

Objectiu de la classe:

- Aprofundir el coneixement dels algorismes de cerca no informada: Cprof, Camp, CCU
- Analitzar el seu funcionament

Problema 1.

Suposeu que tenim un agent que és un robot que es mou per l'habitació de la figura i ha de buscar un camí per anar entre el lloc on es troba i el lloc on ha de fer alguna tasca.

Suposeu que el robot només es pot moure en vertical i en horitzontal, i que les posicions en negre són obstacles per on no pot passar.

19	20	21	22	23	24
13	14		16	17	
7	8	9		11	12
1		3	4	5	6 R

- Com representaríeu el problema?, què són els nodes?, què són les branques?
- Quin és el factor de ramificació del problema?
- Quins algorismes de Cerca no Informada faríeu servir.
- Proposeu possibles criteris d'èxit amb els que podríem avaluar el comportament de l'agent
- Dibuixeu l'arbre de tots els possibles camins que pot seguir des del punt on es troba (R) fins a la posició 21. Construïu tot l'arbre com a mínim fins a profunditat 7.

Problema 1. Solució

a) Com representaríeu el problema?, què són els nodes?, què són les branques?

NODE = Posició del Robot

BRANCA= Moviment del robot

b) Quin és el factor de ramificació del problema?

b=4 (els quatre possibles moviments en horitzontal i vertical)

c) Proposeu possibles criteris d'èxit amb els que podríem avaluar el comportament de l'agent

- El nombre de rajoles per les que ha passat abans d'arribar a la solució, o bé la distància recorreguda.
- Despesa d'energia en el camí fins a l'objectiu
- ...

d) Dibuixeu l'arbre de possibles camins que pot seguir des del punt on es troba (R) fins a la posició 21. Construïu tot l'arbre com a mínim fins a profunditat 7.

... A la pàgina següent

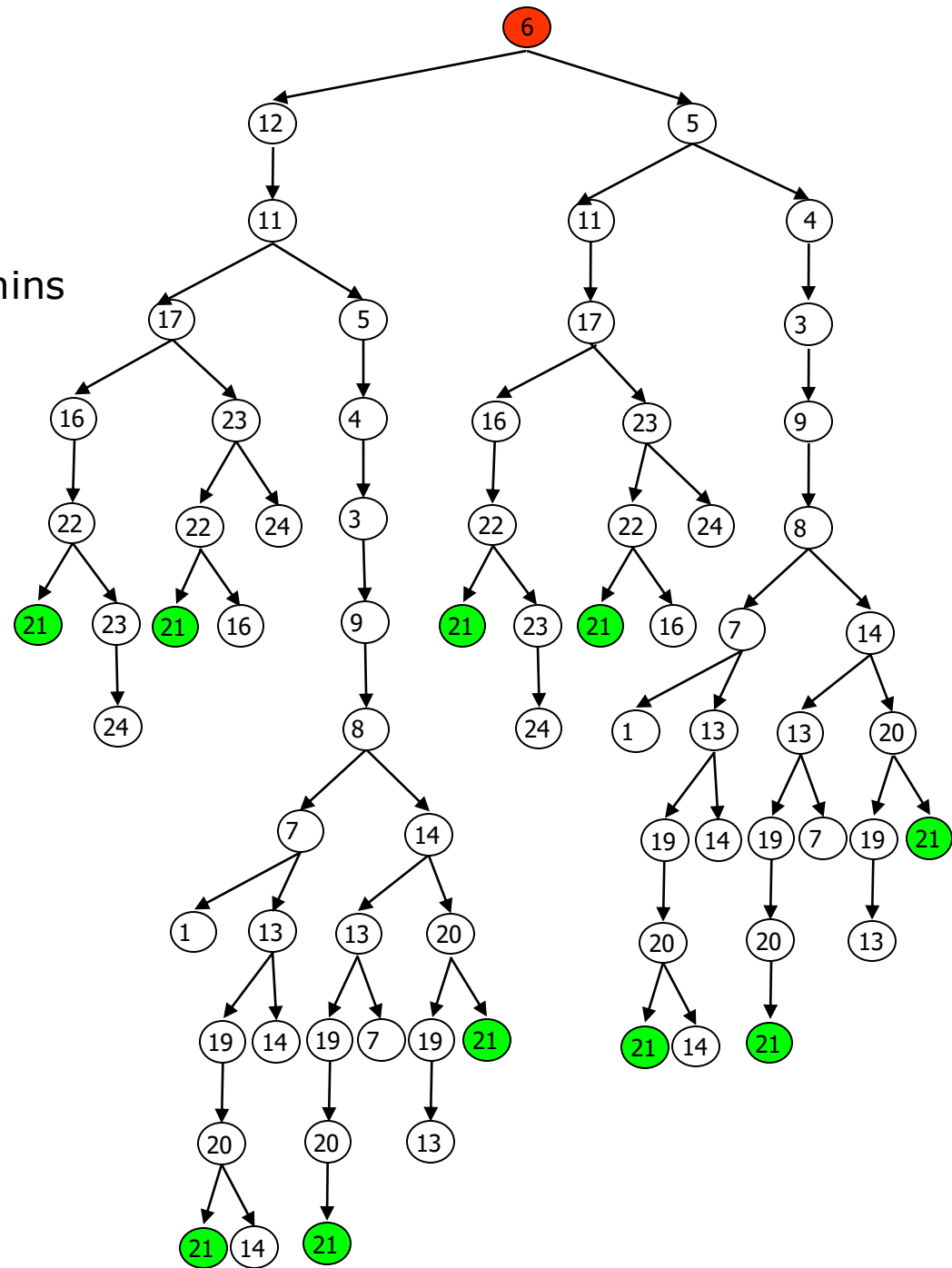
Cerca no informada

Problema 1. Solució

d) Dibuixeu l'arbre de possibles camins que pot seguir des del punt on es troba (R) fins a la posició 21. (1)

Nota important: que un camí arribi a un node al qual ja hi ha arribat un camí anterior, no significa que el sub-arbre que penja d'aquest node sigui una còpia del sub-arbre que penja del mateix node d'un camí anterior.

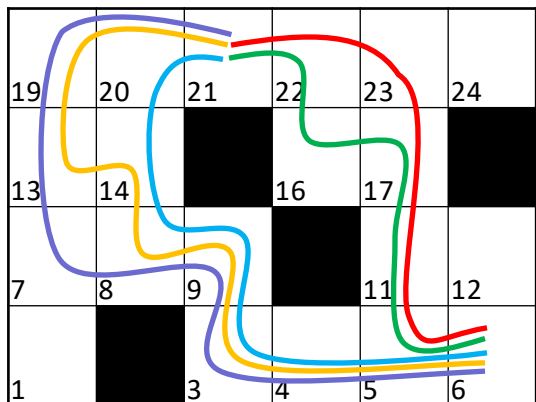
Els cicles que es poden generar per un sub-arbre i per l'altre poden ser diferents.



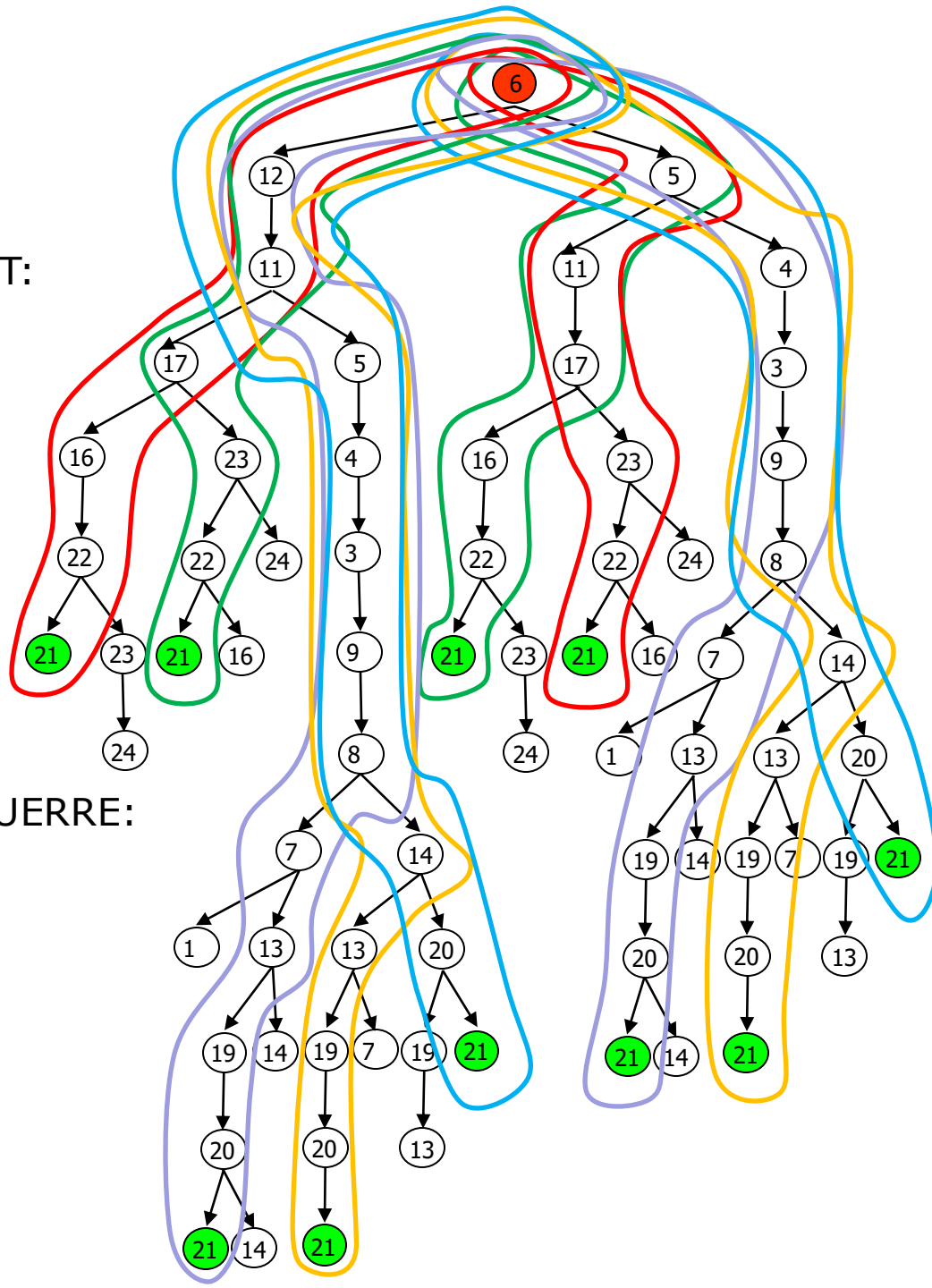
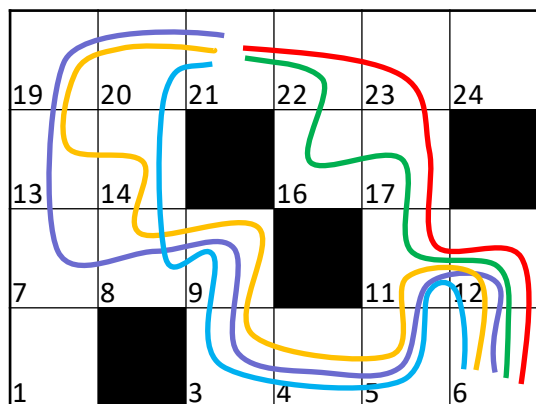
Cerca no informada

Problema 1. Solució

Camins solució al SUB-ARBRE DRET:



Camins solució al SUB-ARBRE ESQUERRE:

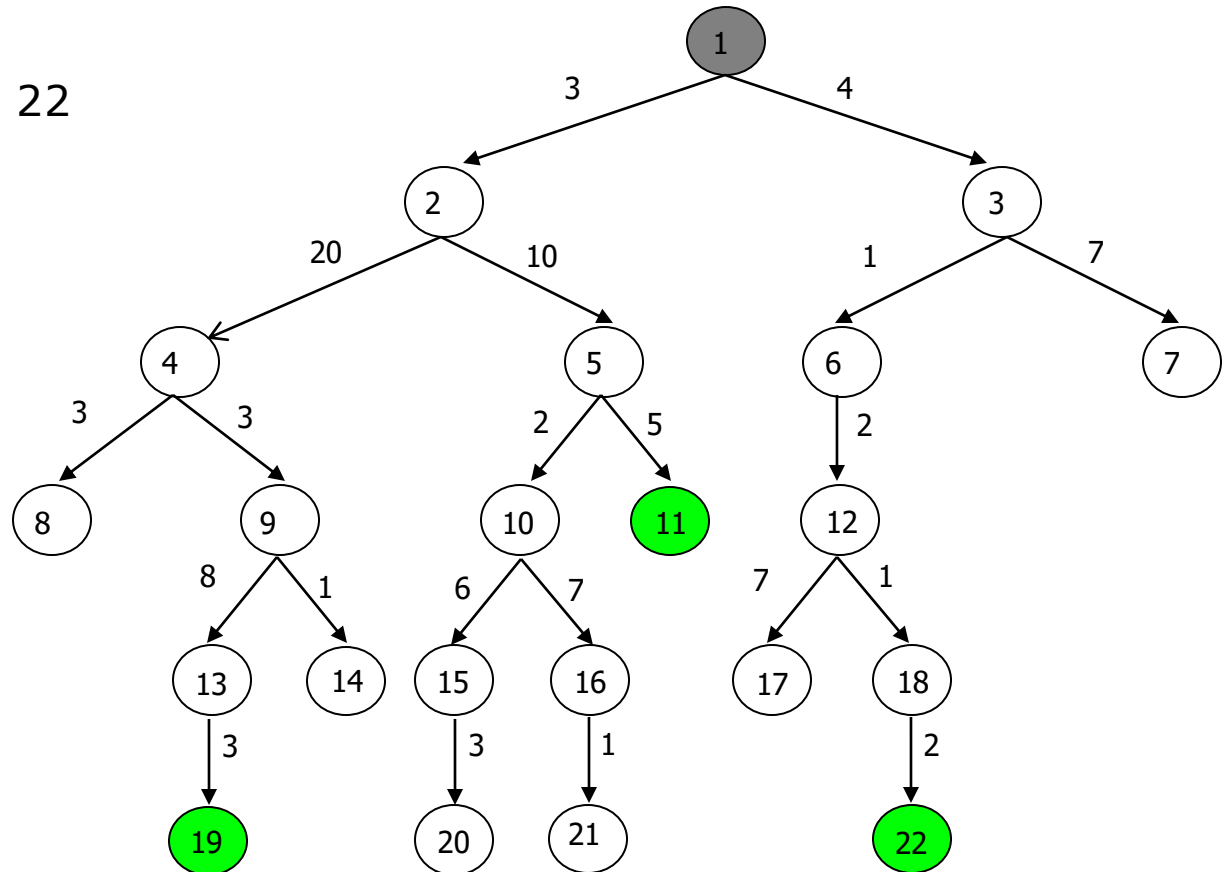


Problema 2.

Suposeu que volem solucionar un problema que requereix generar el següent arbre semàntic amb els costos associats a cada branca:

Estat Inicial: 1

Estats Objectiu: 19, 11 i 22



a) Apliqueu els següents **algorismes de cerca**:

1. Cerca en amplada
2. Cerca em profunditat
3. Cerca de cost uniforme

indiqueu **quina solució donarà cada algorisme** i escriviu les **llistes de camins** tal com faria l'algorisme fins arribar a una solució.

Exemple:

Iteració 1: [[1]]

Iteració 2: [[2,1], [3,1]] ...

a) Apliqueu els següents **algorisme de cerca en amplada**

Iteració 0: [[1]]

Iteració 1: [[2,1], [3,1]]

Iteració 2: [[3,1], [4,2,1], [5,2,1]]

Iteració 3: [[4,2,1], [5,2,1], [6,3,1], [7,3,1]]

Iteració 4: [[5,2,1], [6,3,1], [7,3,1], [8,4,2,1], [9,4,2,1]]

Iteració 5: [[6,3,1], [7,3,1], [8,4,2,1], [9,4,2,1], [10,5,2,1], [11,5,2,1]]

Iteració 6: [[7,3,1], [8,4,2,1], [9,4,2,1], [10,5,2,1], [11,5,2,1], [12,6,3,1]]

Iteració 7: [[8,4,2,1], [9,4,2,1], [10,5,2,1], [11,5,2,1], [12,6,3,1]]

Iteració 8: [[9,4,2,1], [10,5,2,1], [11,5,2,1], [12,6,3,1]]

Iteració 9: [[10,5,2,1], [11,5,2,1], [12,6,3,1], [13,9,4,2,1], [14,9,4,2,1]]

Iteració 10: [[11,5,2,1], [12,6,3,1], [13,9,4,2,1], [14,9,4,2,1],
[15,10,5,2,1], [16,10,5,2,1]]

Iteració 11: FINAL la solució és [11,5,2,1]

a) Apliqueu els següents **algorisme de cerca en profunditat**

Iteració 0: [[1]]

Iteració 1: [[2,1], [3,1]]

Iteració 2: [[4,2,1], [5,2,1], [3,1]]

Iteració 3: [[8,4,2,1], [9,4,2,1], [5,2,1], [3,1]]

Iteració 4: [[9,4,2,1], [5,2,1], [3,1]]

Iteració 5: [[13,9,4,2,1], [14,9,4,2,1], [5,2,1], [3,1]]

Iteració 6: [[19,13,9,4,2,1], [14,9,4,2,1], [5,2,1], [3,1]]

Iteració 7: FINAL la solució és [19,13,9,4,2,1]

a) Apliqueu els següents **algorisme de cerca de cost uniforme**

Nota preliminar: per representar el cost del camí l'introduïm al final d'una llista que té com a primer element el camí i com a segon element el seu cost. Això implicaria alguns canvis a l'algorisme vist a classe.

Iteració 0: [[[1],0]]

Iteració 1: [[[2,1],3], [[3,1],4]]

Iteració 2: [[[3,1],4], [[5,2,1],13], [[4,2,1],23]]

Iteració 3: [[[6,3,1],5], [[7,3,1],11], [[5,2,1],13], [[4,2,1],23]]

Iteració 4: [[[12,6,3,1],7], [[7,3,1],11], [[5,2,1],13], [[4,2,1],23]]

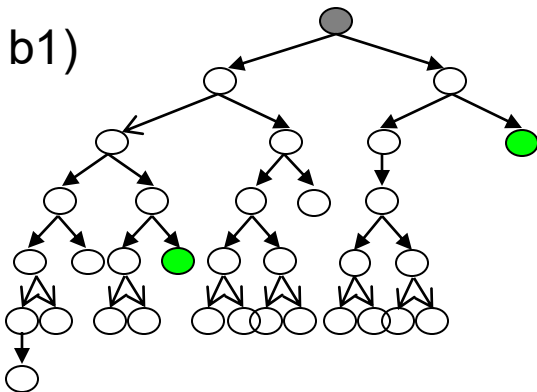
Iteració 5: [[[18,12,6,3,1],8], [[7,3,1],11], [[5,2,1],13],
[[17,12,6,3,1],14], [[4,2,1],23]]

Iteració 6: [[[22,18,12,6,3,1],10], [[7,3,1],11], [[5,2,1],13],
[[17,12,6,3,1],14], [[4,2,1],23]]

Iteració 7: FINAL la solució és [22,18,12,6,3,1] amb cost 10

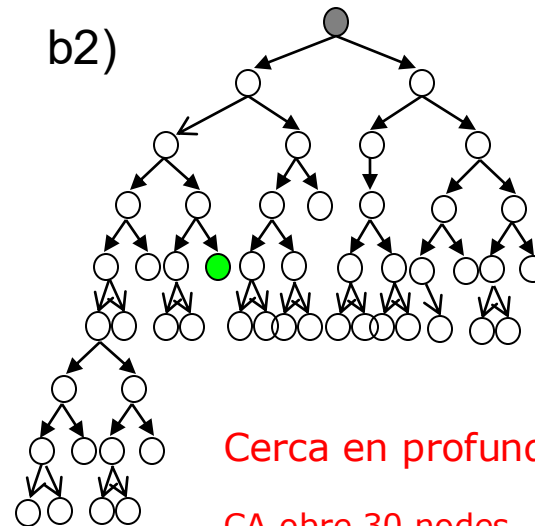
b) Discutiu i responeu les qüestions que teniu a continuació:

1. Quins algorismes garanteixen que sempre arribaran a una solució si aquesta existeix? **C. en prof (si no existeixen camins infinits), C. en amplada, C de cost uniforme**
2. Per problemes que generen els arbres següents, quin algorisme arribarà més ràpid a una solució en cada cas? I quin trigarà més?, podem comptabilitzar el temps en termes de nodes oberts.



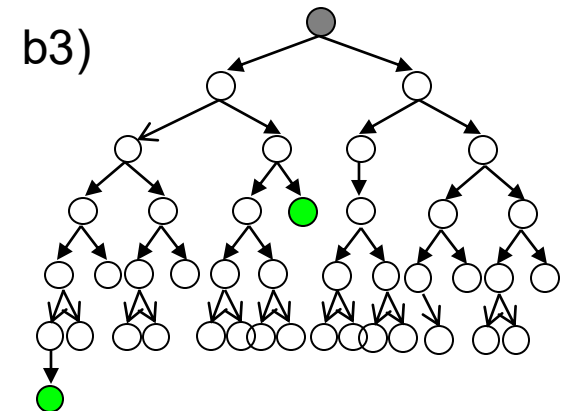
Cerca en amplada

CA obre 12 nodes
CP obre 16 nodes



Cerca en profunditat

CA obre 30 nodes
CP obre 25 nodes



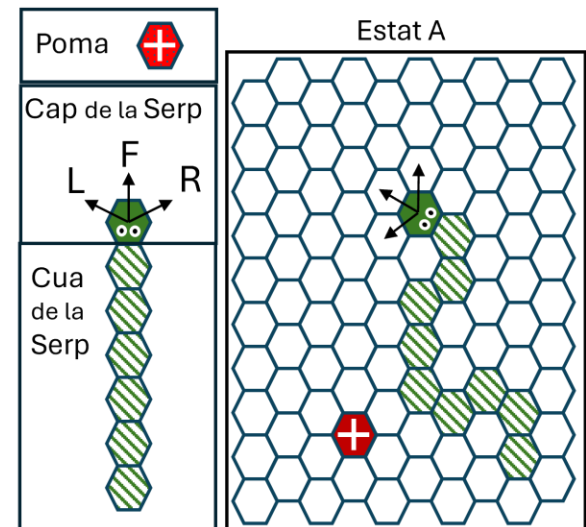
Cerca en profunditat

CA obre 20 nodes
CP obre 12 nodes

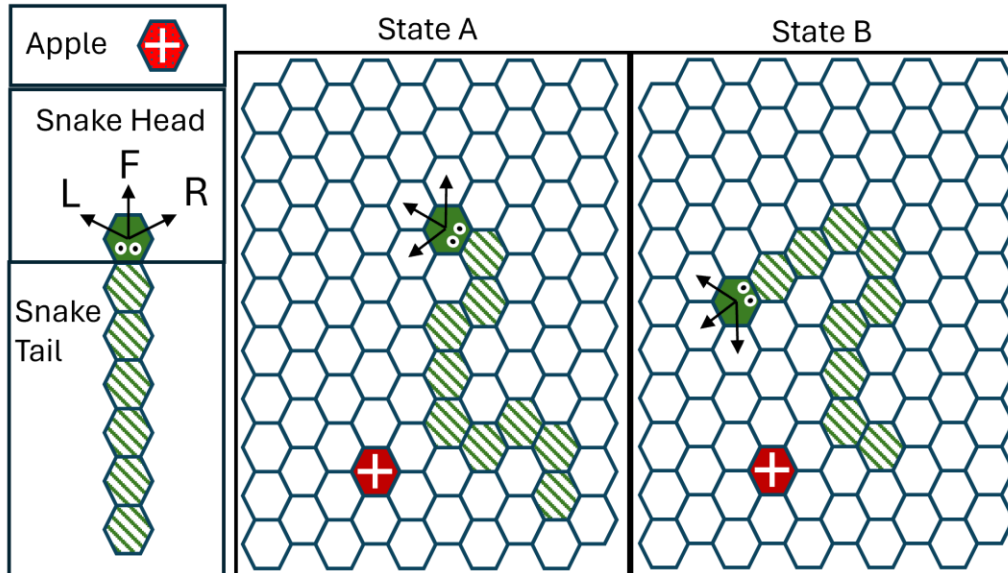
Problema 3.

Suposem que us demanen que programeu un **Agent autònom que sigui capaç de jugar a SNAKE**. L'Snake és un joc individual en el que hi ha una serp que es mou en un taulell arrossegant la seva cua per les caselles on va passant el cap. Té com a objectiu menjar el màxim número de pomes en el mínim temps possible. Les pomes apareixen en posicions aleatòries del taulell no ocupades per la serp. Quan la serp es menja una poma, aleshores apareix la següent poma, i així successivament. Cada vegada que es menja una poma, la longitud de la cua de la serp s'incrementa en una casella al final. En aquest cas tenim les següents particularitats:

- **Les caselles del taulell són hexàgons.** La distància entre els centres de dos hexàgons veïns és 1 i la serp sempre passa pel centre de totes les caselles. Veure un exemple del taulell a la figura (Estat A).
- **Moviments de la serp:** El cap de la serp pot fer 3 tipus de moviments L, F i R (veure figura). El cap no pot anar a una casella que estigui ocupada per la seva cua. Cada vegada que fa un moviment amb el cap, les caselles de la cua segueixen el moviment de la seva casella anterior per mantenir la cua tota connectada. Totes les caselles de la cua passen per on ha passat el cap. La serp sempre sap la posició de la poma.



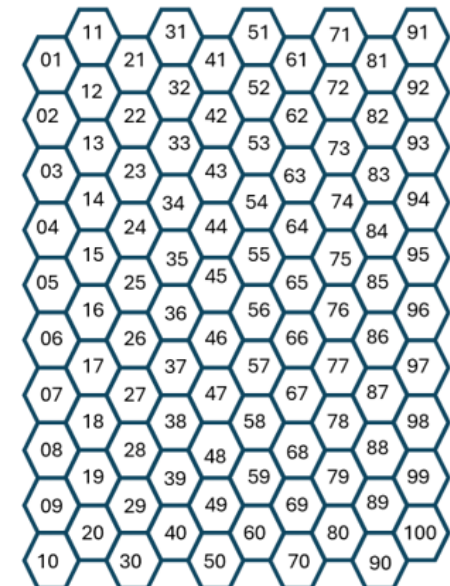
Exemple: En aquesta figura mostrem un exemple de com són els moviments entre dos estats del taulell: A i B. **Estat B** és com estarà el taulell després que la serp hagi fet 3 moviments: L (esquerra), F (endavant), F (endavant), a partir de l'**Estat A**:



Així és com enumerarem les caselles del taulell per poder aplicar els algorismes de cerca.



Enumeració Taulell



a) Considerant que l'algorisme que controla la serp aplica la funció SNAKE de la dreta. Contesteu les següents preguntes:

- Indica quins algorismes de cerca vists fins ara es poden aplicar per implementar la **CERCA** de la funció SNAKE donada:
 - Cerca en profunditat (Sí / No)
 - Cerca en amplada (Sí / No)
 - Cerca de Cost Uniforme (Sí / No)
- Què són els NODES? *Les caselles del taulell*
- Què són les BRANQUES? *Els moviments entre caselles veïnes*
- Què és l'ESTAT INICIAL? *Posició del cap de la Serp*
- Què és l'ESTAT OBJECTIU? *Posició de la Poma*
- Quin és el factor de Ramificació (b)? *3*

b) Suposant que la funció **expand** sempre ens retorna els camins possibles en el següent ordre: L, F, R, i considerant que l'enumeració de les caselles del taulell venen donades com a la figura de la dreta. Digues quina serà la llista de camins que construirà la **Cerca en profunditat**, després de dues iteracions de l'algorisme la serp i la poma estan en el taulell com a **l'Estat A**

Llista^{it=0} -> [[54]]

Llista^{it=1} -> [[43,54],[53,54],[63,54]]

Llista^{it=2} -> [[34,43,54],[33,43,54],[42,43,54],[53,54],[63,54]]

Funció SNAKE(taulell)

Per i=1 fins a ∞ Fer

- camí=CERCA(Pserp,Ppoma)
- Menja_POMA(taulell, camí)

FPer

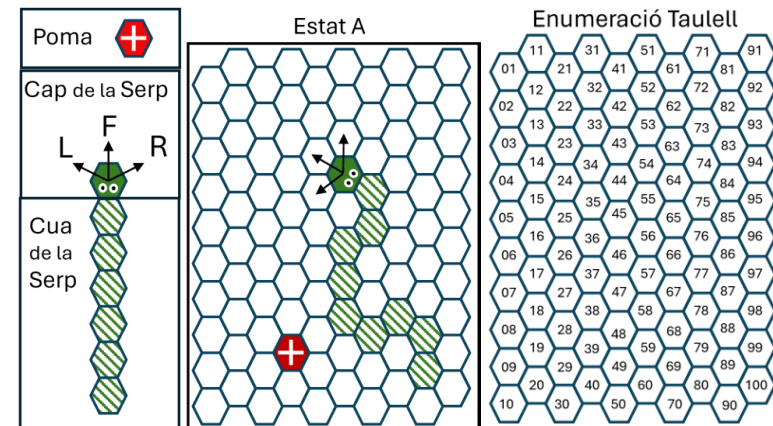
Ffunció

Pserp: posició del cap

Ppoma: posició de la poma

Menja_POMA: Actualitza la posició

de la serp després de fer el camí donat.

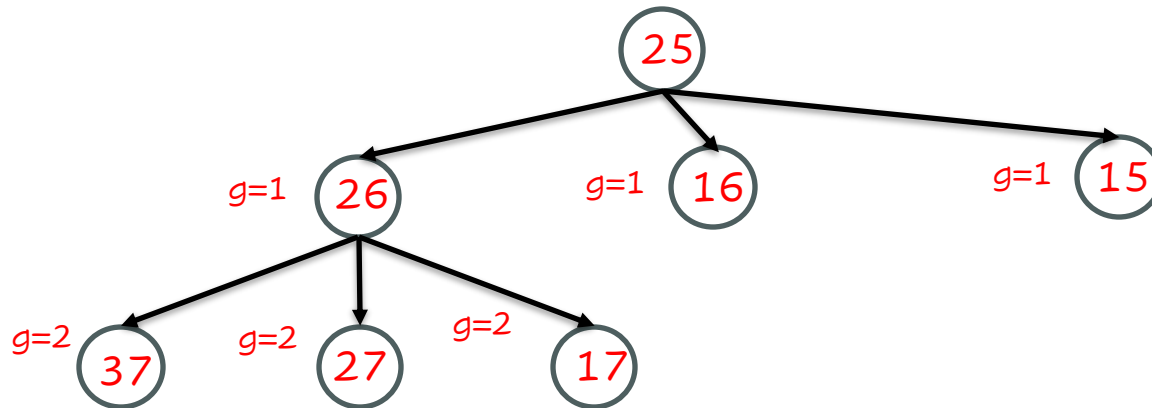


c) Contesta les següents preguntes considerant que substituïm la CERCA de la funció SNAKE per una **Cerca de Cost Uniforme (CCU)**:

- Què serà la funció g de l'algorisme CCU?

L'únic cost que té sentit és el que ens pot donar el camí més curt, per tant el cost és la distància entre caselles (distància entre caselles veïnes=1)

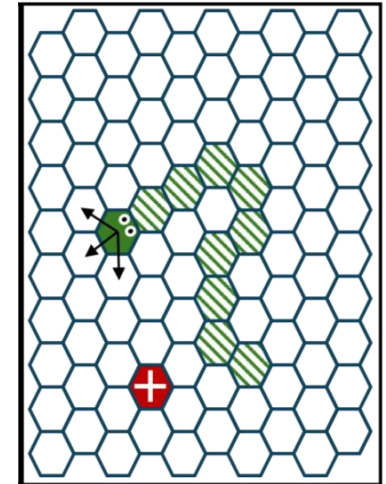
- Dibuixa l'arbre que haurà construït aquest algorisme després de les dues primeres iteracions si partim de la situació de l'Estat B (en cas d'igualtat de costos agafem la de més a l'esquerra),



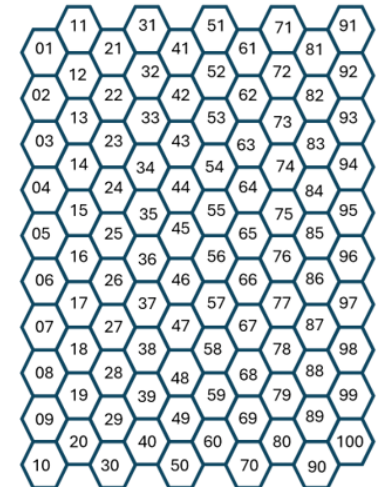
- Indica la llista que representa el camí solució final després de totes les iteracions d'aquest algorisme:

[[39, 38, 37, 26, 25], $g=4$]

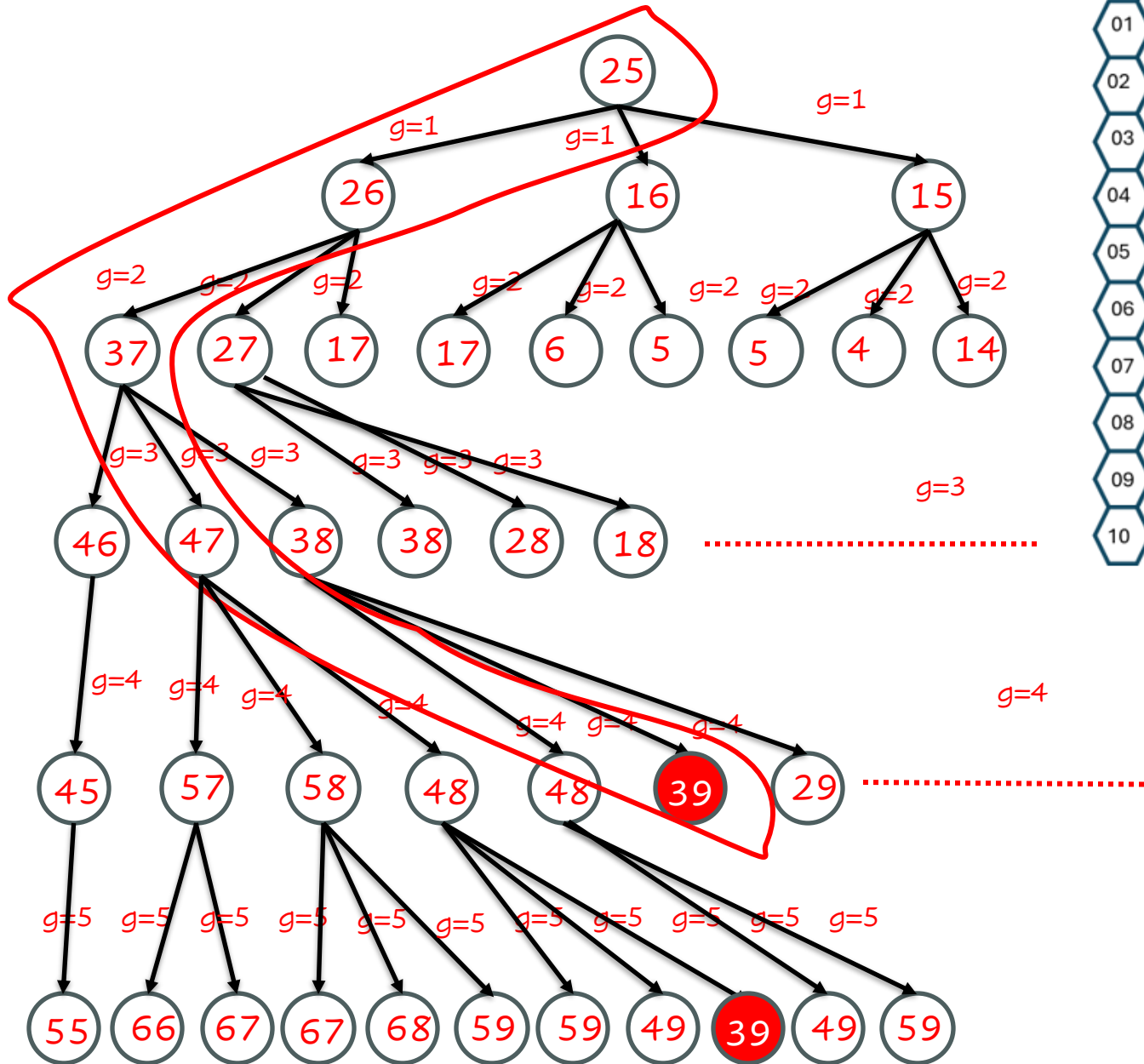
Estat B



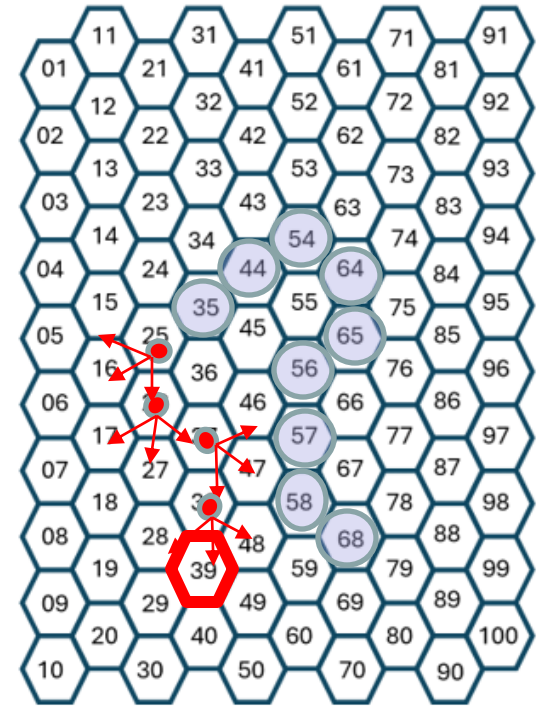
Enumeració Taulell



Arbre construit per l'algorisme CCU



Enumeració Taulell



Estat B

